



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

**INFORME DE LABORATORIO POO MATRIZ, ÁREAS Y
PERÍMETROS — UNIDAD II**

Curso: Programación II

Docente:

- Breno Solim Luque Zúñiga

Ramirez Ajalla, Angie Belen (2024016759)

**Tacna – Perú
2026**

INDICE

I. INFORMACIÓN GENERAL	3
Objetivos	3
Equipos materiales programas y recursos utilizados	3
II. MARCO TEÓRICO	3
III. PROCEDIMIENTO	4
IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS	4
CONCLUSIONES	19
RECOMENDACIONES	20
WEBGRAFIA	21

INFORME DE LABORATORIO POO MATRIZ, ÁREAS Y PERÍMETROS

I. INFORMACIÓN GENERAL

Objetivos

- Aplicar los conceptos de Programación Orientada a Objetos mediante el desarrollo de una aplicación en C# utilizando Windows Forms.
- Diseñar una interfaz gráfica que permita realizar la multiplicación de matrices ingresando filas, columnas y valores numéricos.
- Crear una clase llamada Matriz que permita almacenar datos, ingresar valores, mostrar valores y realizar la multiplicación de matrices.
- Desarrollar una segunda interfaz gráfica para calcular.
- Aplicar herencia, encapsulamiento y polimorfismo mediante una clase base Figura y clases derivadas como Cuadrado, Rectangulo, Triangulo y Circulo.
- Validar los datos ingresados por el usuario para evitar errores durante la ejecución.

Equipos materiales programas y recursos utilizados

- Visual Studio
- Lenguaje de programación C#.
- Windows Forms
- .NET Framework
- Controles gráficos como Label, TextBox, Button, ComboBox y Panel.

II. MARCO TEÓRICO

Programación Orientada a Objetos

La Programación Orientada a Objetos es un paradigma basado en clases y objetos que permite organizar mejor el código mediante propiedades y métodos.

Encapsulamiento

El encapsulamiento consiste en proteger los datos internos de una clase utilizando atributos privados y métodos públicos. Esto permite que la información no sea modificada directamente desde cualquier parte del programa.

Herencia

La herencia permite crear una clase nueva a partir de una clase base. La clase derivada puede reutilizar características y métodos de la clase principal, además de agregar sus propios comportamientos.

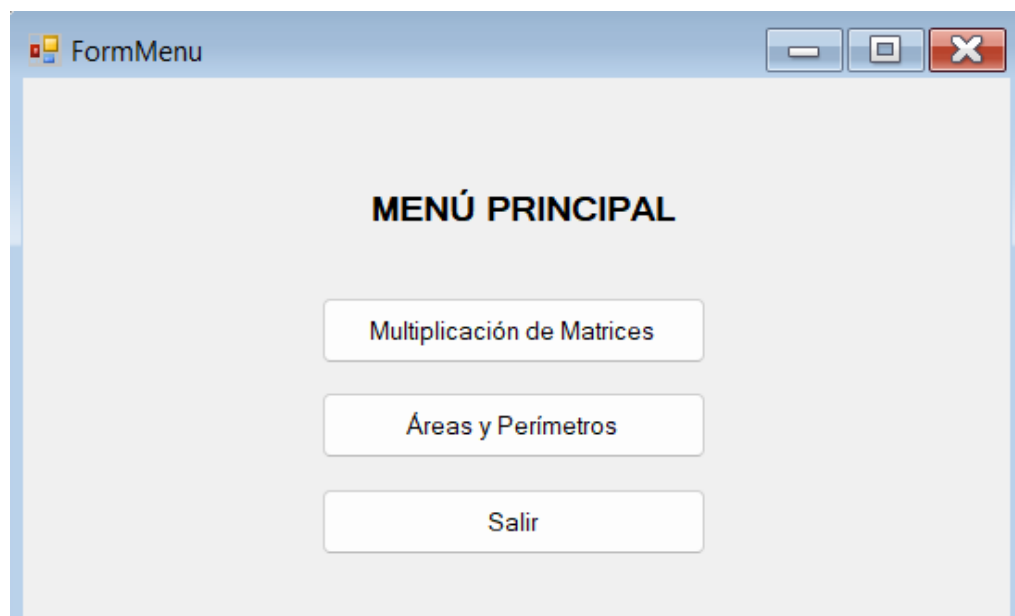
Polimorfismo

El polimorfismo permite que diferentes clases utilicen métodos con el mismo nombre, pero con comportamientos distintos. Esto es útil cuando varias clases comparten una misma estructura, pero necesitan realizar cálculos diferentes.

III. PROCEDIMIENTO

3.1 Ventana de menú principal

Para iniciar el proyecto se creó una ventana principal llamada FormMenu. Esta ventana permite acceder a los dos ejercicios desarrollados en el laboratorio: multiplicación de matrices y cálculo de áreas y perímetros.



3.2 Código del menú principal

Para el funcionamiento del menú principal se programaron los botones correspondientes. El botón de matrices abre el formulario FormMatrices, el botón de figuras abre el formulario FormFiguras y el botón salir cierra completamente la aplicación.

```
1  using System;
2  using System.Windows.Forms;
3
4  namespace P00_Matrices_Ramirez
5  {
6      3 referencias
7      public partial class FormMenu : Form
8      {
9          1 referencia
10         public FormMenu()
11         {
12             InitializeComponent();
13         }
14
15         1 referencia
16         private void btnMatrices_Click(object sender, EventArgs e)
17         {
18             FormMatrices ventana = new FormMatrices();
19             ventana.ShowDialog();
20         }
21
22         1 referencia
23         private void btnFiguras_Click(object sender, EventArgs e)
24         {
25             FormFiguras ventana = new FormFiguras();
26             ventana.ShowDialog();
27         }
28
29         1 referencia
30         private void btnSalir_Click(object sender, EventArgs e)
31         {
32             Application.Exit();
33         }
34     }
35 }
```

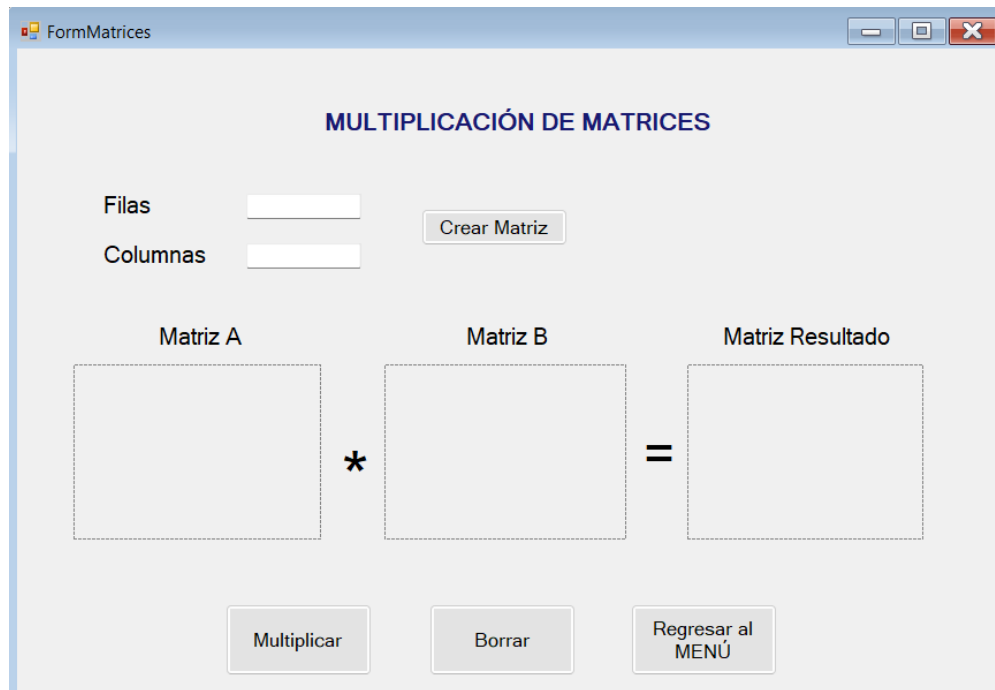
3.3 Código de inicio del programa

El archivo Program.cs fue configurado para que el sistema inicie mostrando primero el menú principal. De esta manera, el usuario puede elegir qué ejercicio desea ejecutar.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Threading.Tasks;
5  using System.Windows.Forms;
6
7  namespace P00_Matrices_Ramirez
8  {
9      0 referencias
10     internal static class Program
11     {
12         [STAThread]
13         0 referencias
14         static void Main()
15         {
16             Application.EnableVisualStyles();
17             Application.SetCompatibleTextRenderingDefault(false);
18             Application.Run(new FormMenu());
19         }
20     }
21 }
```

3.4 Ventana de multiplicación de matrices

Se diseñó el formulario FormMatrices, el cual permite ingresar la cantidad de filas y columnas para crear dos matrices. El formulario cuenta con tres paneles: uno para la Matriz A, otro para la Matriz B y otro para la Matriz Resultado.



The screenshot shows a Windows-style application window titled "FormMatrices". The main content area has a light gray background and is titled "MULTIPLICACIÓN DE MATRICES" in blue text. Below the title, there are two input fields labeled "Filas" and "Columnas", each followed by a small gray button labeled "Crear Matriz". Below these inputs, there are three large dashed rectangular boxes arranged horizontally. The first box is labeled "Matriz A", the second "Matriz B", and the third "Matriz Resultado". Between the first and second boxes is a multiplication symbol (*), and between the second and third boxes is an equals sign (=). At the bottom of the window, there are three gray buttons: "Multiplicar", "Borrar", and "Regresar al MENÚ".

3.5 Código del formulario de matrices

En el formulario de matrices se programó la creación dinámica de cajas de texto dentro de los paneles. Estas cajas permiten ingresar los valores de cada matriz. El botón **Multiplicar** lee los valores ingresados, crea objetos de la clase Matriz, valida si la multiplicación es posible y muestra el resultado final.

```
1  using System;
2  using System.Drawing;
3  using System.Windows.Forms;
4
5  namespace P00_Matrices_Ramirez
6  {
7      4 referencias
8      public partial class FormMatrices : Form
9      {
10         // Arreglos para guardar las cajas de texto de cada matriz
11         TextBox[,] cajaA;
12         TextBox[,] cajaB;
13         TextBox[,] cajaR;
14
15         1 referencia
16         public FormMatrices()
17         {
18             InitializeComponent();
19         }
20
21         1 referencia
22         private void btnCrear_Click(object sender, EventArgs e)
23         {
24             try
25             {
26                 // Leer filas y columnas ingresadas
27                 int fil = int.Parse(txtFil.Text);
28                 int col = int.Parse(txtCol.Text);
29
30                 // Validar datos mayores que cero
31                 if (fil <= 0 || col <= 0)
32                 {
33                     MessageBox.Show(
34                         "Ingrese filas y columnas mayores que cero.",
35                         "Datos no válidos",
36                         MessageBoxButtons.OK,
37                         MessageBoxIcon.Warning
38                     );
39                     return;
40                 }
41
42                 // Con este diseño ambas matrices usan la misma cantidad de filas y columnas.
43                 // Se recomienda usar matrices cuadradas para que se puedan multiplicar.
44                 if (fil != col)
45                 {
46                     MessageBox.Show(
47                         "No se puede multiplicar con esas dimensiones.\n\n" +
48                         "Para multiplicar matrices se debe cumplir:\n\n" +
49                         "Columnas de la Matriz A = Filas de la Matriz B.\n\n" +
50                         "Con este diseño, la Matriz A y la Matriz B se crean con el mismo tamaño.\n" +
51                         "Si ingresas 2 filas y 3 columnas, se crea:\n\n" +
52                         "Matriz A: 2 x 3\n" +
53                         "Matriz B: 2 x 3\n\n" +
54                         "Eso no se puede multiplicar porque las columnas de A son 3 y las filas de B son 2.\n\n" +
55                         "Usa matrices cuadradas como 2x2, 3x3 o 4x4.",
56                         "Multiplicación no válida",
57                         MessageBoxButtons.OK,
58                         MessageBoxIcon.Warning
59                     );
60                     return;
61                 }
62
63                 // Limpiar paneles antes de crear nuevas matrices
64                 pnlA.Controls.Clear();
65             }
66             catch { }
67         }
68     }
69 }
```

```

59     pnlB.Controls.Clear();
60     pnlR.Controls.Clear();
61
62     // Crear cajas para Matriz A y Matriz B
63     crearCajas(pnlA, fil, col, out cajaA, false);
64     crearCajas(pnlB, fil, col, out cajaB, false);
65
66     cajaR = null;
67 }
68 catch
69 {
70     MessageBox.Show(
71         "Ingrese solo números válidos en filas y columnas.",
72         "Error",
73         MessageBoxButtons.OK,
74         MessageBoxIcon.Error
75     );
76 }
77 }
78 1 referencia
79 private void btnMult_Click(object sender, EventArgs e)
80 {
81     try
82     {
83         // Validar que primero se hayan creado las matrices
84         if (cajaA == null || cajaB == null)
85         {
86             MessageBox.Show(
87                 "Primero debe crear las matrices.",
88                 "Aviso",
89                 MessageBoxButtons.OK,
90                 MessageBoxIcon.Information
91             );
92             return;
93         }
94         // Leer los datos de los TextBox y convertirlos en objetos Matriz
95         Matriz A = leerMatriz(cajaA);
96         Matriz B = leerMatriz(cajaB);
97         // Validar la regla de multiplicación de matrices
98         if (!A.sePuedeMultiplicar(B))
99         {
100             MessageBox.Show(
101                 "No se puede realizar la multiplicación.\n\n" +
102                 "Regla:\n" +
103                 "Columnas de la Matriz A = Filas de la Matriz B.\n\n" +
104                 "Si no se cumple esa condición, no existe una matriz resultado válida.",
105                 "Dimensiones incorrectas",
106                 MessageBoxButtons.OK,
107                 MessageBoxIcon.Warning
108             );
109             return;
110         }
111         // Multiplicar usando el método de la clase Matriz
112         Matriz R = A.multiplicar(B);
113
114         // Limpiar resultado anterior
115         pnlR.Controls.Clear();
116
117         // Crear cajas para mostrar la matriz resultado
118         crearCajas(pnlR, R.Fil, R.Col, out cajaR, true);
119
120         // Mostrar valores del resultado
121         for (int i = 0; i < R.Fil; i++)
122         {
123             for (int j = 0; j < R.Col; j++)
124             {
125                 cajaR[i, j].Text = R.verValor(i, j).ToString("0.##");
126             }
127         }
128     }
129     catch
130     {
131         MessageBox.Show(

```



```

131         "Verifique que todos los espacios de las matrices tengan números válidos.",
132         "Error",
133         MessageBoxButtons.OK,
134         MessageBoxIcon.Error
135     );
136     }
137 }
138 // 1 referencia
139 private void btnBorrar_Click(object sender, EventArgs e)
140 {
141     // Limpiar filas y columnas
142     txtFil.Clear();
143     txtCol.Clear();
144
145     // Limpiar paneles
146     pnlA.Controls.Clear();
147     pnlB.Controls.Clear();
148     pnlR.Controls.Clear();
149
150     // Vaciar arreglos
151     cajaA = null;
152     cajaB = null;
153     cajaR = null;
154
155     // Regresar cursor al primer TextBox
156     txtFil.Focus();
157 }
158 // 1 referencia
159 private void btnMenu_Click(object sender, EventArgs e)
160 {
161     // Cierra este formulario
162     this.Close();
163 }
164 // Método para leer los valores de las cajas y convertirlos en objeto Matriz
165 // 2 referencias
166 private Matriz leerMatriz(TextBox[,] cajas)
167 {
168     int fil = cajas.GetLength(0);
169     int col = cajas.GetLength(1);
170
171     Matriz m = new Matriz(fil, col);
172
173     for (int i = 0; i < fil; i++)
174     {
175         for (int j = 0; j < col; j++)
176         {
177             double valor = double.Parse(cajas[i, j].Text);
178             m.ponerValor(i, j, valor);
179         }
180     }
181     return m;
182 }
183 // Método para crear cajas de texto dentro de un panel
184 // 3 referencias
185 private void crearCajas(Panel pnl, int fil, int col, out TextBox[,] cajas, bool soloLectura)
186 {
187     cajas = new TextBox[fil, col];
188
189     int ancho = 45;
190     int alto = 28;
191     int espacio = 8;
192     // Calcular tamaño total para centrar las cajas en el panel
193     int totalAncho = col * ancho + (col - 1) * espacio;
194     int totalAlto = fil * alto + (fil - 1) * espacio;
195
196     int inicioX = (pnl.Width - totalAncho) / 2;
197     int inicioY = (pnl.Height - totalAlto) / 2;

```

```

195
196     if (inicioX < 5)
197     {
198         inicioX = 5;
199     }
200     if (inicioY < 5)
201     {
202         inicioY = 5;
203     }
204     for (int i = 0; i < fil; i++)
205     {
206         for (int j = 0; j < col; j++)
207         {
208             TextBox txt = new TextBox();
209             txt.Width = ancho;
210             txt.Height = alto;
211             txt.Left = inicioX + j * (ancho + espacio);
212             txt.Top = inicioY + i * (alto + espacio);
213             txt.TextAlign = HorizontalAlignment.Center;
214             txt.ReadOnly = soloLectura;
215             txt.Font = new Font("Microsoft Sans Serif", 10, FontStyle.Regular);
216             if (soloLectura)
217             {
218                 txt.BackColor = Color.White;
219             }
220             pnl.Controls.Add(txt);
221             cajas[i, j] = txt;
222         }
223     }
224 }
225
226 }

```

3.6 Clase Matriz

Se creó la clase Matriz, la cual contiene atributos privados para almacenar filas, columnas y datos. Además, contiene métodos públicos para ingresar valores, obtener valores, validar la multiplicación y multiplicar matrices. Esta clase aplica encapsulamiento porque los datos internos no se modifican directamente desde el formulario.

```

1  namespace P00_Matrices_Ramirez
2  {
3      12 referencias
4      public class Matriz
5      {
6          // Atributos privados para encapsulamiento
7          private int fil;
8          private int col;
9          private double[,] datos;
10
11         // Constructor
12         2 referencias
13         public Matriz(int fil, int col)
14         {
15             this.fil = fil;
16             this.col = col;
17             datos = new double[fil, col];
18         }
19     }
20 }

```

```

18      2 referencias
19      public int Fil
20      {
21          |   get { return fil; }
22      }
23
24      2 referencias
25      public int Col
26      {
27          |   get { return col; }
28      }
29
30      // Método para ingresar valores
31      2 referencias
32      public void ponerValor(int i, int j, double valor)
33      {
34          |   datos[i, j] = valor;
35      }
36
37      // Método para mostrar valores
38      1 referencia
39      public double verValor(int i, int j)
40      {
41          |   return datos[i, j];
42      }
43
44      // Método para validar multiplicación
45      1 referencia
46      public bool sePuedeMultiplicar(Matriz otra)
47      {
48          |   return this.col == otra.fil;
49      }
50
51      // Método para multiplicar matrices
52      1 referencia
53      public Matriz multiplicar(Matriz otra)
54      {
55          |   Matriz r = new Matriz(this.fil, otra.col);
56
57          |   for (int i = 0; i < this.fil; i++)
58          |   {
59              |   for (int j = 0; j < otra.col; j++)
60              |   {
61                  |   double suma = 0;
62
63                  |   for (int k = 0; k < this.col; k++)
64                  |   {
65                      |   suma += this.datos[i, k] * otra.datos[k, j];
66                  |   }
67
68                  |   r.ponerValor(i, j, suma);
69              |   }
70          |   }
71
72          |   return r;
73      }
74
75      }
76
77      }

```

3.7 Ejecución de la multiplicación de matrices

Para comprobar el funcionamiento, se ingresaron valores en la Matriz A y Matriz B. Luego se presionó el botón Multiplicar, obteniendo la matriz resultada en el panel correspondiente.

MULTIPLICACIÓN DE MATRICES

Filas

Columnas

Matriz A				Matriz B				Matriz Resultado		
1	1	2		2	1	2		7	10	6
2	1	1	*	1	3	2	=	7	8	7
2	1	1		2	3	1		7	8	7

3.8 Ventana de áreas y perímetros

Se diseñó el formulario FormFiguras, el cual permite seleccionar una figura geométrica. Las figuras consideradas fueron: Cuadrado, Rectángulo, Triángulo y Círculo.

CÁLCULO DE ÁREA Y PERÍMETRO

Selección de figura

Figura:

Datos de la figura

Dato 1:

Dato 2:

Dato 3:

Resultados

Área:

Perímetro:

3.9 Código del formulario de áreas y perímetros

En el formulario de figuras se programó el ComboBox para mostrar las figuras disponibles. Según la figura seleccionada, el sistema cambia los nombres de los datos solicitados y oculta las cajas que no se utilizan. El botón Calcular crea el objeto correspondiente y muestra el área y perímetro. El botón Limpiar reinicia los datos del formulario.

```
1  using System;
2  using System.Windows.Forms;
3
4  namespace P00_Matrices_Ramirez
5  {
6      4 referencias
7      public partial class FormFiguras : Form
8      {
9          1 referencia
10         public FormFiguras()
11         {
12             InitializeComponent();
13         }
14         1 referencia
15         private void FormFiguras_Load(object sender, EventArgs e)
16         {
17             // Agregar figuras al ComboBox
18             cboFigura.Items.Add("Cuadrado");
19             cboFigura.Items.Add("Rectángulo");
20             cboFigura.Items.Add("Triángulo");
21             cboFigura.Items.Add("Círculo");
22
23             // Evita que el usuario escriba dentro del ComboBox
24             cboFigura.DropDownStyle = ComboBoxStyle.DropDownList;
25
26             // Estado inicial
27             limpiarDatos();
28         }
29         1 referencia
30         private void cboFigura_SelectedIndexChanged(object sender, EventArgs e)
31         {
32             // Limpiar cajas cada vez que se cambia de figura
33             txtDato1.Clear();
34             txtDato2.Clear();
35             txtDato3.Clear();
36
37             lblArea.Text = "Área:";
38             lblPer.Text = "Perímetro:";
39
40             // Ocultar todos primero
41             lblDato1.Visible = true;
42             txtDato1.Visible = true;
43             lblDato2.Visible = false;
44             txtDato2.Visible = false;
45             lblDato3.Visible = false;
46             txtDato3.Visible = false;
47
48             // Cambiar los datos pedidos según la figura
49             if (cboFigura.Text == "Cuadrado")
50             {
51                 lblDato1.Text = "Lado:";
52             }
53             else if (cboFigura.Text == "Rectángulo")
54             {
55                 lblDato1.Text = "Base:";
56                 lblDato2.Text = "Altura:";
57
58                 lblDato2.Visible = true;
59                 txtDato2.Visible = true;
60             }
61         }
62     }
63 }
```

```

56     }
57     else if (cboFigura.Text == "Triángulo")
58     {
59         lblDato1.Text = "Base:";
60         lblDato2.Text = "Altura:";
61         lblDato3.Text = "Lado:";
62         lblDato2.Visible = true;
63         txtDato2.Visible = true;
64         lblDato3.Visible = true;
65         txtDato3.Visible = true;
66     }
67     else if (cboFigura.Text == "Círculo")
68     {
69         lblDato1.Text = "Radio:";
70     }
71 }
72 1 referencia
73 private void btnCalc_Click(object sender, EventArgs e)
74 {
75     try
76     {
77         if (cboFigura.SelectedIndex == -1)
78         {
79             MessageBox.Show("Seleccione una figura.");
80             return;
81         }
82         Figura figura = null;
83         if (cboFigura.Text == "Cuadrado")
84         {
85             double lado = double.Parse(txtDato1.Text);
86
87             if (lado <= 0)
88             {
89                 MessageBox.Show("El lado debe ser mayor que cero.");
90                 return;
91             }
92
93             figura = new Cuadrado(lado);
94         }
95         else if (cboFigura.Text == "Rectángulo")
96         {
97             double baseRect = double.Parse(txtDato1.Text);
98             double altura = double.Parse(txtDato2.Text);
99
100             if (baseRect <= 0 || altura <= 0)
101             {
102                 MessageBox.Show("La base y la altura deben ser mayores que cero.");
103                 return;
104             }
105
106             figura = new Rectangulo(baseRect, altura);
107         }
108         else if (cboFigura.Text == "Triángulo")
109         {
110             double baseTri = double.Parse(txtDato1.Text);
111             double altura = double.Parse(txtDato2.Text);
112             double lado = double.Parse(txtDato3.Text);
113
114             if (baseTri <= 0 || altura <= 0 || lado <= 0)
115             {
116                 MessageBox.Show("La base, altura y lado deben ser mayores que cero.");
117                 return;
118             }
119
120             figura = new Triangulo(baseTri, altura, lado);
121         }
122         else if (cboFigura.Text == "Círculo")
123         {
124             double radio = double.Parse(txtDato1.Text);
125

```

```

126         if (radio <= 0)
127         {
128             MessageBox.Show("El radio debe ser mayor que cero.");
129             return;
130         }
131
132         figura = new Circulo(radio);
133     }
134
135     lblArea.Text = "Área: " + figura.CalcularArea().ToString("0.00");
136     lblPer.Text = "Perímetro: " + figura.CalcularPerimetro().ToString("0.00");
137 }
138 catch
139 {
140     MessageBox.Show("Ingrese solo números válidos.");
141 }
142 }
143
144 1 referencia
145 private void btnLimpiar_Click(object sender, EventArgs e)
146 {
147     LimpiarDatos();
148 }
149 1 referencia
150 private void btnMenu_Click(object sender, EventArgs e)
151 {
152     this.Close();
153 }
154 2 referencias
155 private void LimpiarDatos()
156 {
157     cboFigura.SelectedIndex = -1;
158
159     txtDato1.Clear();
160     txtDato2.Clear();
161     txtDato3.Clear();
162     lblDato1.Text = "Dato 1:";
163     lblDato2.Text = "Dato 2:";
164     lblDato3.Text = "Dato 3:";
165     lblDato1.Visible = true;
166     txtDato1.Visible = true;
167     lblDato2.Visible = true;
168     txtDato2.Visible = true;
169     lblDato3.Visible = true;
170     txtDato3.Visible = true;
171     lblArea.Text = "Área:";
172     lblPer.Text = "Perímetro:";
173 }
174 }

```

3.10 Clase base Figura

Se creó la clase abstracta Figura, la cual contiene una propiedad general llamada Nombre y dos métodos abstractos: CalcularArea () y CalcularPerimetro (). Esta clase funciona como base.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace POO_Matrices_Ramirez
8  {
9      9 referencias
9      public abstract class Figura
10     {
11         // Propiedad general de la figura
12         4 referencias
12         public string Nombre { get; set; }
13
14         // Métodos que cada figura debe sobrescribir
15         5 referencias
15         public abstract double CalcularArea();
16
17         5 referencias
17         public abstract double CalcularPerimetro();
18     }
19 }
```

3.11 Clases derivadas de Figura

Se crearon las clases Cuadrado, Rectangulo, Triangulo y Circulo. Cada clase hereda de Figura y sobrescribe los métodos CalcularArea () y CalcularPerimetro (). Cada figura realiza sus propios cálculos según sus fórmulas correspondientes.

Clase Cuadrado

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace POO_Matrices_Ramirez
8  {
9      2 referencias
9      public class Cuadrado : Figura
10     {
11         private double lado;
12
13         1 referencia
13         public Cuadrado(double lado)
14         {
15             this.lado = lado;
16             this.Nombre = "Cuadrado";
17         }
18
19         2 referencias
19         public override double CalcularArea()
20         {
21             return lado * lado;
22         }
23
24         2 referencias
24         public override double CalcularPerimetro()
25         {
26             return 4 * lado;
27         }
28     }
29 }
```


Clase Rectángulo

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace POO_Matrices_Ramirez
8  {
9      2 referencias
10     public class Rectangulo : Figura
11     {
12         private double baseRectangulo;
13         private double altura;
14
15         1 referencia
16         public Rectangulo(double baseRectangulo, double altura)
17         {
18             this.baseRectangulo = baseRectangulo;
19             this.altura = altura;
20             this.Nombre = "Rectángulo";
21         }
22
23         2 referencias
24         public override double CalcularArea()
25         {
26             return baseRectangulo * altura;
27         }
28
29         2 referencias
30         public override double CalcularPerimetro()
31         {
32             return 2 * (baseRectangulo + altura);
33         }
34     }
35 }
```

Clase Triángulo

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace POO_Matrices_Ramirez
8  {
9      2 referencias
10     public class Triangulo : Figura
11     {
12         private double baseTriangulo;
13         private double altura;
14         private double lado;
15
16         1 referencia
17         public Triangulo(double baseTriangulo, double altura, double lado)
18         {
19             this.baseTriangulo = baseTriangulo;
20             this.altura = altura;
21             this.lado = lado;
22             this.Nombre = "Triángulo";
23         }
24
25         2 referencias
26         public override double CalcularArea()
27         {
28             return (baseTriangulo * altura) / 2;
29         }
30
31         2 referencias
32         public override double CalcularPerimetro()
33         {
34             return 3 * lado;
35         }
36     }
37 }
```

Clase Círculo

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace P00_Matrices_Ramirez
8  {
9      2 referencias
10     public class Círculo : Figura
11     {
12         private double radio;
13
14         1 referencia
15         public Círculo(double radio)
16         {
17             this.radio = radio;
18             this.Nombre = "Círculo";
19         }
20
21         2 referencias
22         public override double CalcularArea()
23         {
24             return Math.PI * radio * radio;
25         }
26
27         2 referencias
28         public override double CalcularPerimetro()
29         {
30             return 2 * Math.PI * radio;
31         }
32     }
33 }
```

3.12 Ejecución del cálculo de áreas y perímetros

Para comprobar el funcionamiento, se seleccionó una figura geométrica, se ingresaron sus medidas y se presionó el botón Calcular. El sistema mostró correctamente el área y perímetro.

The image displays two screenshots of a web application titled "CÁLCULO DE ÁREA Y PERÍMETRO".

Left Screenshot:

- Selección de figura:** A dropdown menu showing "Cuadrado".
- Datos de la figura:** A text input field labeled "Lado:" with the value "4".
- Buttons:** "Calcular" and "Limpiar".
- Resultados:** A box displaying "Área: 16,00" and "Perímetro: 16,00".

Right Screenshot:

- Selección de figura:** A dropdown menu showing "Rectángulo".
- Datos de la figura:** Two text input fields: "Base:" with the value "3" and "Altura:" with the value "4".
- Buttons:** "Calcular" and "Limpiar".
- Resultados:** A box displaying "Área: 12,00" and "Perímetro: 14,00".

CÁLCULO DE ÁREA Y PERÍMETRO

Selección de figura
Figura: Triángulo

Datos de la figura
Base: 3
Altura: 6
Lado: 2

Calcular Limpiar

Resultados
Área: 9,00
Perímetro: 6,00

CÁLCULO DE ÁREA Y PERÍMETRO

Selección de figura
Figura: Círculo

Datos de la figura
Radio: 3

Calcular Limpiar

Resultados
Área: 28,27
Perímetro: 18,85

IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

¿Qué indican los resultados?

- La aplicación permitió ejecutar correctamente los dos ejercicios solicitados: multiplicación de matrices y cálculo de áreas y perímetros.
- El menú principal permitió acceder de forma ordenada a cada formulario del sistema.
- En el formulario de matrices, se logró generar dinámicamente las matrices A y B mediante cajas de texto dentro de paneles.
- La clase Matriz permitió organizar la lógica de la multiplicación, aplicando encapsulamiento mediante atributos privados y métodos públicos.
- La aplicación validó que las matrices puedan multiplicarse correctamente, considerando que las columnas de la Matriz A deben coincidir con las filas de la Matriz B.
- En el formulario de figuras, se logró seleccionar una figura geométrica, ingresar sus medidas y mostrar el área y perímetro correspondiente.

¿Qué se ha encontrado?

- Los métodos permiten reutilizar instrucciones, como ingresar valores, obtener datos, multiplicar matrices, calcular áreas y calcular perímetros.

- El encapsulamiento protege los datos internos de las clases, evitando que sean modificados directamente desde el formulario.
- La herencia permitió que las clases de figuras geométricas compartan una estructura común mediante la clase base Figura.
- El polimorfismo permitió que cada figura tenga sus propios cálculos de área y perímetro usando los mismos métodos generales.

CONCLUSIONES

- Se logró desarrollar una aplicación en C# Windows Forms aplicando Programación Orientada a Objetos para resolver los ejercicios propuestos en el laboratorio.
- Se implementó un menú principal que permite acceder al formulario de multiplicación de matrices y al formulario de áreas y perímetros.
- En el primer ejercicio se creó la clase Matriz, aplicando encapsulamiento, constructores y métodos para ingresar valores, obtener datos y multiplicar matrices.
- En el segundo ejercicio se creó la clase base Figura y las clases derivadas Cuadrado, Rectangulo, Triangulo y Circulo, aplicando herencia y polimorfismo.
- La interfaz gráfica permitió al usuario ingresar datos, ejecutar operaciones, visualizar resultados y limpiar la información de manera sencilla.

RECOMENDACIONES

- Mantener el código separado en clases para que el proyecto sea más ordenado y fácil de modificar.
- Usar nombres claros en formularios, botones, cajas de texto, variables y métodos para facilitar la lectura del programa.
- Validar siempre los datos ingresados por el usuario para evitar errores durante la ejecución.
- Probar la aplicación con diferentes valores para verificar que los cálculos sean correctos.
- Mejorar progresivamente el diseño visual de los formularios para que la interfaz sea más clara y agradable.

WEBGRAFIA

Microsoft. (2025). Windows Forms overview. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/>

Microsoft. (2025). Classes and objects in C#. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/classes>

Microsoft. (2025). Inheritance in C#. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/inheritance>

Microsoft. (2025). Polymorphism in C#. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism>